



Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia
Licenciatura em Engenharia de Sistemas Informáticos

Disciplina: Integração de Sistemas de Informação

Ano letivo 2024/2025

Trabalho Prático

Gestão de Máquinas de Vending

Alunos:

21151 – João Machado

20484 – Vítor Sá

Professor: Luís Gonzaga Martins Ferreira

Barcelos

dezembro/ 2024

Índice

1.	Descrição breve do cenário a ser estudado no projeto	4
2.	Modelo de Dados.....	5
	Estrutura Geral do Modelo:.....	5
	Relacionamentos entre Tabelas	7
3.	Descrição das Principais Operações a Implementar (Endpoints)	8
4.	Arquitetura	10
	Operações a Implementar em Web Services SOAP (XML).....	10
	Operações a Implementar em Web API RESTful	10
	Implementação e Acesso à Base de Dados	11
	Serviços externos consumidos	11
	Benefícios da Implementação.....	12
5.	Desenvolvimento à data do relatório.....	13
	Operações Implementadas em Web Services SOAP (XML).....	13
	Operações Implementadas em Web API RESTful	14
	Desenvolvimento Frontend	16
6.	Próximos passos	24
7.	Conclusão.....	26
8.	Bibliografia.....	27

Índice de figuras

Figura 1 - Ecrã MAUI Categoria.....	21
-------------------------------------	----

1. Descrição breve do cenário a ser estudado no projeto

O projeto explora o cenário de gestão de máquinas de vending, abrangendo desde o armazenamento e reposição de produtos até à gestão de transações e inventário em tempo real. Este cenário é frequentemente encontrado em empresas que operam redes de máquinas de venda automática, oferecendo produtos como bebidas, snacks e outros itens de conveniência em locais estratégicos como escritórios, escolas, hospitais e transportes públicos.

O objetivo é desenvolver uma solução eficiente e escalável que permita:

1. Gestão Centralizada - Controlar remotamente o inventário e as vendas de máquinas espalhadas por diferentes regiões.
2. Melhoria na Experiência do Cliente - Garantir que os produtos estejam sempre disponíveis e manter as máquinas em bom estado de funcionamento, melhorando a satisfação dos clientes.
3. Criar uma interface de simples utilização compatível com múltiplas plataformas.

Este projeto também aborda desafios como a integração de APIs externas, uma estrutura de dados expansível e acessível online, e a criação de uma aplicação multiplataformas.

2. Modelo de Dados

O modelo de dados é uma parte essencial de qualquer sistema de informação, pois define como as informações são organizadas, armazenadas e acedidas. No contexto do projeto de gestão de máquinas de vending, o modelo de dados foi projetado para refletir a complexidade e a relação entre diferentes entidades, garantindo escalabilidade, integridade e facilidade de uso.

Estrutura Geral do Modelo:

O modelo de dados deste sistema foi desenvolvido utilizando uma abordagem relacional, implementada na base de dados PostgreSQL sendo o seu host a plataforma CockroachDB. Ele inclui as seguintes tabelas principais:

1. Categoria (Category) - Representa as categorias dos produtos vendidos, como "Bebidas", "Snacks" e "Doces".

- Campos principais:
 - i. id (chave primária): Identificador único da categoria.
 - ii. name: Nome da categoria.
 - iii. description: Descrição detalhada da categoria.

2. Produto (Product) - Representa os produtos vendidos pelas máquinas. Relaciona-se com a tabela Categoria através de uma chave estrangeira (category_id).

- Campos principais:
 - i. id (chave primária): Identificador único do produto.
 - ii. name: Nome do produto.
 - iii. description: Descrição detalhada do produto.
 - iv. price: Preço do produto.
 - v. category_id: Referência à categoria do produto.
 - vi. is_active: Indica se o produto está ativo ou não.

3. Máquina de Vending (Vending Machine) - Representa as máquinas de vending que armazenam os produtos.

- Campos principais:

- i. id (chave primária): Identificador único da máquina.
- ii. location: Localização física da máquina.
- iii. description: Descrição detalhada da máquina.

4. Inventário (Inventory) - Representa os produtos armazenados em cada máquina de vending e suas respectivas quantidades. Relaciona-se com as tabelas Máquina de Vending e Produto.

- Campos principais:

- i. id (chave primária): Identificador único do registro de inventário.
- ii. machine_id: Referência à máquina onde o produto está armazenado.
- iii. product_id: Referência ao produto armazenado.
- iv. quantity: Quantidade do produto disponível.

5. Transações (Transactions) - Regista as vendas realizadas pelas máquinas de vending. Relaciona-se com as tabelas Máquina de Vending e Produto.

- Campos principais:

- i. id (chave primária): Identificador único da transação.
- ii. machine_id: Referência à máquina onde a transação ocorreu.
- iii. product_id: Referência ao produto vendido.
- iv. quantity: Quantidade do produto vendido.
- v. total_price: Valor total da transação.
- vi. transaction_date: Data e horário da transação.

6. Utilizadores (Users) - Representa os utilizadores do sistema, incluindo administradores e operadores.

- Campos principais:

- i. id (chave primária): Identificador único do utilizador.
- ii. username: Nome de utilizador único.
- iii. password: Senha criptografada do utilizador.
- iv. role: Função do utilizador (e.g., "admin" ou "operator").

Relacionamentos entre Tabelas

O modelo de dados estabelece os seguintes relacionamentos:

- Categoria e Produto: Uma categoria pode ter vários produtos (relacionamento 1:N).
- Produto e Inventário: Um produto pode estar em várias máquinas, mas cada registo de inventário é único para uma combinação de produto e máquina (relacionamento N:M implementado através da tabela Inventory).
- Máquina de Vending e Inventário: Cada máquina pode ter vários produtos em seu inventário (relacionamento 1:N).
- Transações e Produto/Máquina de Vending: Cada transação está associada a uma máquina e a um produto específico (relacionamento N:M).

3. Descrição das Principais Operações a Implementar (Endpoints)

Abaixo estão descritos os endpoints principais que deverão ser implementados no sistema, agrupados pelas funcionalidades relacionadas:

1. Categoria (Category):

- GET /api/category: Retorna a lista de categorias.
- GET /api/category/{id}: Retorna uma categoria pelo seu ID.
- POST /api/category: Cria uma nova categoria.
- PUT /api/category/{id}: Atualiza os dados de uma categoria existente.
- DELETE /api/category/{id}: Remove uma categoria.

2. Produto (Product):

- GET /api/product: Retorna a lista de produtos.
- GET /api/product/{id}: Retorna um produto pelo seu ID.
- POST /api/product: Cria um novo produto associado a uma categoria.
- PUT /api/product/{id}: Atualiza os dados de um produto existente.
- DELETE /api/product/{id}: Remove um produto.

3. Máquina de Vending (Vending Machine):

- GET /api/vending-machine: Retorna a lista de máquinas de vending.
- GET /api/vending-machine/{id}: Retorna uma máquina de vending pelo seu ID.
- POST /api/vending-machine: Cria uma nova máquina de vending.
- PUT /api/vending-machine/{id}: Atualiza os dados de uma máquina existente.
- DELETE /api/vending-machine/{id}: Remove uma máquina de vending.

4. Inventário (Inventory):

- GET /api/inventory: Retorna o inventário completo.
- GET /api/inventory/{machineId}: Retorna o inventário de uma máquina específica.
- POST /api/inventory: Adiciona ou atualiza produtos no inventário de uma máquina.

5. Transações (Transactions):

- GET /api/transactions: Retorna o histórico de transações.
- GET /api/transactions/{id}: Retorna os detalhes de uma transação específica.
- POST /api/transactions: Regista uma nova transação.

6. Utilizadores (Users):

- GET /api/users: Retorna a lista de utilizadores.

- GET /api/users/{id}: Retorna os detalhes de um utilizador.
- POST /api/users: Cria um novo utilizador.
- PUT /api/users/{id}: Atualiza os dados de um utilizador.
- DELETE /api/users/{id}: Remove um utilizador.

4. Arquitetura

Operações a Implementar em Web Services SOAP (XML)

O serviço SOAP foi projetado para assegurar uma gestão completa e centralizada de todas as tabelas da base de dados do sistema de gestão de máquinas de vending. A implementação destas operações visa garantir a integridade dos dados, promovendo a interoperabilidade com sistemas externos.

Os principais objetivos das operações SOAP são os seguintes:

- Permitir a inserção de novos registos nas tabelas da base de dados (e.g., produtos, categorias, máquinas, utilizadores, entre outros).
- Realizar a atualização de informações existentes nas entidades do sistema.
- Proporcionar a consulta estruturada de dados, garantindo respostas consistentes e adequadas às necessidades do sistema.
- Facilitar a eliminação de registos, assegurando a integridade referencial dos dados armazenados.

Operações a Implementar em Web API RESTful

O serviço RESTful foi concebido para fornecer uma interface flexível e acessível, destinada ao consumo por aplicações front-end e por outros serviços, promovendo a integração eficiente e a interação em tempo real com os dados do sistema de gestão de máquinas de vending. Assim como o SOAP, as operações RESTful têm como objetivo assegurar uma gestão completa das entidades do sistema, garantindo integridade e acessibilidade aos dados.

Os principais objetivos das operações RESTful são os seguintes:

- Permitir a inserção de novos registos nas tabelas da base de dados, como produtos, categorias, máquinas, utilizadores, entre outros.
- Realizar a atualização de dados existentes, possibilitando alterações em atributos como nomes, preços, localizações e funções de utilizadores.
- Disponibilizar a consulta de informações estruturadas, permitindo a recuperação de dados relevantes de forma eficiente e adaptada às necessidades do sistema.
- Proporcionar a eliminação de registos, garantindo que a integridade e as relações entre os dados sejam preservadas.

Implementação e Acesso à Base de Dados

A base de dados é gerida de forma diferenciada entre os serviços SOAP e a API RESTful. No caso do SOAP, os comandos SQL são utilizados diretamente no código, garantindo um controlo detalhado e eficiente sobre as operações de manipulação e consulta de dados. Esta abordagem é ideal para operações mais formais e sensíveis, onde o controlo direto é essencial.

Já na API RESTful, as operações sobre a base de dados são mediadas pela própria arquitetura da aplicação. Embora não tenha sido especificado o uso de um ORM (Object-Relational Mapping), o acesso é estruturado de forma a garantir a integração segura e eficiente com o sistema. A validação e a formatação dos dados continuam a ser processadas na aplicação antes da interação com a base de dados, assegurando a consistência e a integridade dos registos. Esta abordagem centralizada permite que a base de dados seja gerida exclusivamente através da API, promovendo a interoperabilidade e a escalabilidade do sistema.

Serviços externos consumidos

1. Conversão de Moedas

O sistema integra a API **FreeCurrencyAPI** para obter taxas de câmbio atualizadas em relação ao euro. Esta funcionalidade permite realizar cálculos de conversão monetária com precisão, suportando transações em diferentes moedas dentro da aplicação.

Detalhes da Implementação

- **Classe de Cliente HTTP:**

- Foi criada uma classe `CurrencyApiClient` para encapsular a comunicação com a API de conversão de moedas. Esta classe utiliza o `HttpClient` para realizar chamadas HTTP ao serviço externo.
- A URL base da API é armazenada como uma propriedade privada na classe, contendo a chave de autenticação e o parâmetro para definir o euro (EUR) como moeda base:

```
1. private readonly string _baseUrl =  
"https://api.freecurrencyapi.com/v1/latest?apikey=fca_live_a9uasdP6yjM3v5UY4hLLHtXmw2CsvhC040STf0pz&base_currency=EUR";  
2.
```

- **Método para Obter Taxas de Câmbio:**

- O método `GetCurrenciesAsync` realiza a chamada ao endpoint da API e processa a resposta:

```
1. public async Task<List<CurrencyData>> GetCurrenciesAsync()  
2. {  
3.     var response = await _httpClient.GetAsync(_baseUrl);  
4.     response.EnsureSuccessStatusCode();
```

```

5.     var json = await response.Content.ReadAsStringAsync();
6.     var apiResponse = JsonConvert.DeserializeObject<CurrencyData>(json);
7.     return new List<CurrencyData> { apiResponse };
8. }
9.

```

- O método:

- Realiza uma requisição GET para obter os dados de câmbio.
- Assegura que a resposta da API seja bem-sucedida com `EnsureSuccessStatusCode()`.
- Converte o JSON retornado em um objeto `CurrencyData` utilizando o `Json.NET` (via `JsonConvert.DeserializeObject()`).

- **Estrutura da Resposta:**

- A resposta da API é mapeada para uma classe `CurrencyData`, que contém os valores de câmbio em relação ao euro. Exemplo:

```

1. {
2.   "data": {
3.     "USD": 1.10,
4.     "GBP": 0.85,
5.     "JPY": 140.25
6.   }
7. }
8.

```

- **Benefícios da Abstração:**

- A classe `CurrencyApiClient` centraliza toda a lógica de comunicação com a API, promovendo reusabilidade e organização do código.
- O sistema pode consumir as taxas de câmbio através do método `GetCurrenciesAsync`, facilitando a integração em outras partes da aplicação.

Benefícios da Implementação

- **Atualizações em Tempo Real:** As taxas de câmbio são obtidas diretamente da API, garantindo que os cálculos estejam alinhados com o mercado atual.
- **Centralização:** A lógica de comunicação com a API está encapsulada, simplificando manutenções e modificações futuras.
- **Flexibilidade:** A aplicação pode consumir os dados de câmbio para diversas finalidades, como conversões em transações ou exibição de valores em diferentes moedas.

5. Desenvolvimento à data do relatório

Operações Implementadas em Web Services SOAP (XML)

O desenvolvimento do serviço SOAP até à data concentra-se na gestão de utilizadores através de um conjunto de funcionalidades fundamentais para garantir a interação eficiente com a base de dados. Estas funcionalidades incluem operações CRUD (Create, Read, Update, Delete) para a tabela de utilizadores, permitindo inserir, consultar, atualizar e eliminar registos diretamente na base de dados alojada na CockroachDB.

O serviço implementado foi estruturado em duas partes principais:

- **IService.cs:** Interface que define os contratos de serviço, especificando os métodos disponíveis, como `testconnection`, `getUsers`, `InsertUser`, `UpdateUser` e `DeleteUser`.
- **Service.cs:** Implementação do serviço, utilizando a biblioteca `Npgsql` para estabelecer a conexão com a base de dados PostgreSQL. Todos os comandos SQL são executados diretamente no código, assegurando maior controlo sobre as operações realizadas.

Entre as funcionalidades implementadas, destaca-se:

- **Testes de Conexão:** O método `testconnection` verifica a disponibilidade da base de dados, garantindo que as operações podem ser executadas corretamente.
- **Gestão de Utilizadores:** Métodos como `getUsers` permitem consultar todos os utilizadores, enquanto `InsertUser`, `UpdateUser` e `DeleteUser` possibilitam a manipulação direta dos dados, assegurando que a aplicação cumpre os requisitos de gestão.

Até ao momento, o foco principal esteve na robustez e eficiência das operações relacionadas à tabela de utilizadores, garantindo uma base sólida para expandir as funcionalidades SOAP em futuras iterações do projeto.

Segue um excerto relevante do código implementado no serviço SOAP, demonstrando a funcionalidade de consulta de utilizadores na base de dados:

```
1. public List<user> getUsers()
2. {
3.     string connectionString = DatabaseConfig.ConnectionString;
4.
5.     try
6.     {
7.         var conn = new NpgsqlConnection(connectionString);
8.         conn.Open();
9.         var cmd = new NpgsqlCommand("SELECT * FROM users", conn);
```

```

10.     var reader = cmd.ExecuteReader();
11.     var users = new List<user>();
12.
13.     while (reader.Read())
14.     {
15.         var usr = new user
16.         {
17.             id = reader.GetInt64(0),
18.             username = reader.GetString(1),
19.             password = reader.GetString(2),
20.             role = (Role)Enum.Parse(typeof(Role), char.ToUpper(reader.GetString(3)[0]) +
reader.GetString(3).Substring(1).ToLower())
21.         };
22.         users.Add(usr);
23.     }
24.
25.     conn.Close();
26.     return users;
27. }
28. catch (Exception e)
29. {
30.     Console.WriteLine(e);
31.     return null;
32. }
33. }
34.

```

Descrição do Funcionamento

1. Conexão com a Base de Dados:

- A conexão com a base de dados é estabelecida utilizando a biblioteca Npgsql, com os parâmetros definidos em DatabaseConfig.ConnectionString.

2. Consulta SQL:

- A query SELECT * FROM users é executada para recuperar todos os registos da tabela users.

3. Leitura de Dados:

- Um NpgsqlDataReader é utilizado para iterar pelos resultados, criando objetos do tipo user que são adicionados a uma lista.

4. Retorno de Dados:

- A lista de utilizadores é retornada ao cliente, garantindo que todas as informações relevantes estão acessíveis para processamento.

Este exemplo reflete a abordagem adotada para garantir eficiência, controle e robustez na execução de operações diretamente sobre a base de dados.

Operações Implementadas em Web API RESTful

O desenvolvimento da API até à data concentrou-se na criação de uma estrutura modular e organizada, garantindo a implementação de funcionalidades essenciais para a gestão das entidades principais do sistema: utilizadores, produtos, categorias e inventário. A API foi desenvolvida utilizando ASP.NET Core e segue uma estrutura baseada em Controllers e Models, como ilustrado na organização

do projeto. Cada entidade possui uma classe modelo que define os seus atributos e restrições, bem como um controlador correspondente que expõe as operações CRUD através de endpoints RESTful.

Por exemplo, o UserController implementa métodos para listar, consultar, adicionar, atualizar e eliminar utilizadores, utilizando o Entity Framework Core para interagir com a base de dados PostgreSQL. O modelo User é anotado com atributos do Data Annotations para mapear os campos da tabela na base de dados, assegurando a validação e consistência dos dados. Esta estrutura é replicada para as restantes entidades (Product, Category e Inventory), permitindo uma gestão completa e centralizada do sistema.

Além disso, a configuração do projeto inclui um ficheiro appsettings.json para armazenar as definições de ligação à base de dados, e a classe DbContext para gerir o contexto da aplicação. A API está devidamente documentada e testada, com endpoints claramente definidos, preparados para integração com sistemas externos e consumo por clientes front-end. Esta base sólida permitirá a expansão do sistema nas próximas etapas do desenvolvimento.

Segue um excerto relevante do código implementado na API, demonstrando a funcionalidade de gestão de utilizadores no UserController:

```
1. [HttpPost]
2. public async Task<ActionResult<User>> PostUser(User user)
3. {
4.     _context.Users.Add(user);
5.     await _context.SaveChangesAsync();
6.
7.     return CreatedAtAction(nameof(GetUser), new { id = user.Id }, user);
8. }
9.
10. [HttpGet("{id}")]
11. public async Task<ActionResult<User>> GetUser(long id)
12. {
13.     var user = await _context.Users.FindAsync(id);
14.
15.     if (user == null)
16.         return NotFound();
17.
18.     return user;
19. }
20.
21. [HttpDelete("{id}")]
22. public async Task<IActionResult> DeleteUser(long id)
23. {
24.     var user = await _context.Users.FindAsync(id);
25.     if (user == null)
26.         return NotFound();
27.
28.     _context.Users.Remove(user);
29.     await _context.SaveChangesAsync();
30.
31.     return NoContent();
32. }
33.
```

Descrição do Funcionamento

1. PostUser:

- Endpoint POST /api/User para criar um novo utilizador.
- Adiciona o utilizador à base de dados e retorna o recurso criado com o método CreatedAtAction.

2. GetUser:

- Endpoint GET /api/User/{id} para consultar um utilizador pelo seu ID.
- Retorna os dados do utilizador, ou um erro 404 Not Found caso o registo não exista.

3. DeleteUser:

- Endpoint DELETE /api/User/{id} para eliminar um utilizador.
- Remove o registo da base de dados e retorna uma resposta 204 No Content se a operação for bem-sucedida.

Este exemplo reflete a simplicidade e eficiência da estrutura da API, demonstrando como as operações CRUD são implementadas de forma consistente em todas as entidades do sistema.

Desenvolvimento Frontend

Até ao momento, foram desenvolvidas duas aplicações frontend em C# para interagir com as APIs implementadas no sistema. A primeira é uma aplicação em consola, criada para testar e validar as funcionalidades expostas pelas APIs, proporcionando um ambiente simples e controlado para verificar o correto funcionamento dos serviços. A segunda é uma aplicação desenvolvida em .NET MAUI (Multiplatform App UI), que oferece uma interface gráfica multiplataforma, permitindo uma interação mais intuitiva com os serviços RESTful e SOAP. Ambas as aplicações foram projetadas para consumir as APIs de forma eficiente, assegurando a funcionalidade do sistema e servindo como base para futuras melhorias na experiência do utilizador.

Aplicação em Consola

A aplicação em consola foi desenvolvida em C# com o objetivo de facilitar a interação com as APIs implementadas, permitindo o consumo tanto de serviços RESTful quanto de serviços SOAP. Esta aplicação foi concebida para servir como uma ferramenta de teste e validação, garantindo a funcionalidade e a integração do sistema com serviços externos e internos. A sua estrutura modular

organiza as operações em menus específicos para diferentes funcionalidades, como gestão de categorias, produtos, utilizadores e conversão de moedas.

Através desta aplicação, é possível:

- Realizar operações CRUD (Create, Read, Update, Delete) nas tabelas do sistema, como Category e Product.
- Integrar com serviços externos, como a FreeCurrencyAPI, para conversão de moedas em tempo real.
- Consumir serviços SOAP, como a gestão de utilizadores, utilizando chamadas definidas no serviço.

Funcionalidades Principais

1. Gestão de Categorias

A aplicação permite realizar operações sobre a tabela de categorias através da API RESTful. As operações incluem:

- Listagem de todas as categorias.
- Consulta de categorias por ID.
- Criação de novas categorias.
- Atualização de categorias existentes.
- Eliminação de categorias.

Exemplo de implementação de uma operação de listagem:

```
1. public async Task<List<Category>> GetCategoriesAsync()
2. {
3.     var response = await _httpClient.GetAsync(_baseUrl);
4.     response.EnsureSuccessStatusCode();
5.
6.     var json = await response.Content.ReadAsStringAsync();
7.     return JsonConvert.DeserializeObject<List<Category>>(json);
8. }
9.
```

2. Gestão de Produtos

Similar à gestão de categorias, a aplicação inclui funcionalidades para a tabela Product, com operações como:

Visualizar todos os produtos ou um produto específico por ID.

- Criar produtos.
- Atualizar detalhes de produtos existentes.

- Eliminar produtos.

Exemplo de criação de um produto:

```
1 1. . public async Task<Product> CreateProductAsync(Product product)
2. 2. {
3. 3.     var json = JsonConvert.SerializeObject(product);
4. 4.     var content = new StringContent(json, Encoding.UTF8, "application/json");
5. 5.
6. 6.     var response = await _httpClient.PostAsync(_baseUrl, content);
7. 7.     response.EnsureSuccessStatusCode();
8. 8.
9. 9.     var responseJson = await response.Content.ReadAsStringAsync();
10. 10.     return JsonConvert.DeserializeObject<Product>(responseJson);
11. 11.
12. }
13.
```

3. Conversão de Moedas

A integração com a API externa FreeCurrencyAPI permite a conversão de valores monetários em tempo real. Os utilizadores podem inserir um valor em euros, e a aplicação apresenta o valor convertido em várias moedas.

Exemplo de integração com a API externa:

```
1. public async Task<List<CurrencyData>> GetCurrenciesAsync()
2. {
3.     var response = await _httpClient.GetAsync(_baseUrl);
4.     response.EnsureSuccessStatusCode();
5.
6.     var json = await response.Content.ReadAsStringAsync();
7.     var apiResponse = JsonConvert.DeserializeObject<CurrencyData>(json);
8.     return new List<CurrencyData> { apiResponse };
9. }
10.
```

4. Gestão de Utilizadores

Através do serviço SOAP implementado, a aplicação permite a gestão de utilizadores, incluindo a listagem de todos os registos presentes na tabela users. Exemplo de implementação:

```
1. private static void UserManagement(ClienteBasico.UserService.ServiceClient ws)
2. {
3.     Console.WriteLine("\n=== User Management ===");
4.     var users = ws.getUsers();
5.     foreach (var user in users)
6.     {
7.         Console.WriteLine($"ID: {user.id}, Name: {user.username}, Role: {user.role}");
8.     }
9. }
10.
```

Estrutura da Aplicação

A aplicação é organizada de forma modular, com classes específicas para cada funcionalidade:

- CategoryApiClient: Para operações relacionadas à tabela Category.
- ProductApiClient: Para operações relacionadas à tabela Product.
- CurrencyApiClient: Para integração com a API de conversão de moedas.
- Menus: Menus interativos estruturados para facilitar a navegação pelas funcionalidades.

Exemplo de menu para a gestão de categorias:

```
1. private static async Task CategoryMenu(CategoryApiClient apiClient)
2. {
3.     while (true)
4.     {
5.         Console.Clear();
6.         Console.WriteLine("=== Category Management Menu ===");
7.         Console.WriteLine("1. View All Categories");
8.         Console.WriteLine("2. View Category by ID");
9.         Console.WriteLine("3. Create a New Category");
10.        Console.WriteLine("4. Update an Existing Category");
11.        Console.WriteLine("5. Delete a Category");
12.        Console.WriteLine("6. Back to Main Menu");
13.        Console.Write("Select an option: ");
14.
15.        var choice = Console.ReadLine();
16.
17.        switch (choice)
18.        {
19.            case "1":
20.                await ViewAllCategories(apiClient);
21.                break;
22.            case "2":
23.                await ViewCategoryById(apiClient);
24.                break;
25.            case "3":
26.                await CreateCategory(apiClient);
27.                break;
28.            case "4":
29.                await UpdateCategory(apiClient);
30.                break;
31.            case "5":
32.                await DeleteCategory(apiClient);
33.                break;
34.            case "6":
35.                return;
36.            default:
37.                Console.WriteLine("Invalid choice. Please try again.");
38.                break;
39.        }
40.
41.        Console.WriteLine("Press any key to return to the Category Management menu...");
42.        Console.ReadKey();
43.    }
44. }
45.
```

Contribuições e Expansão

A aplicação em consola fornece um ambiente robusto e prático para validar o funcionamento das APIs desenvolvidas e integrar serviços externos. Esta solução demonstra:

- A utilização eficiente de chamadas RESTful e SOAP.

- A capacidade de consumo de serviços externos, como APIs de conversão de moedas.
- A modularidade e escalabilidade da arquitetura.

Futuramente, a aplicação poderá ser expandida para incluir mais funcionalidades e interações com outras tabelas e serviços, consolidando o sistema como uma ferramenta abrangente e eficaz.

Aplicação em MAUI

A aplicação desenvolvida em **.NET MAUI** (Multi-platform App UI) foi projetada para oferecer uma interface gráfica moderna e multiplataforma, capaz de funcionar em dispositivos Android, iOS, Windows e Mac. Esta aplicação interage exclusivamente com a API interna do sistema, focando-se, até ao momento, na gestão da tabela Category. A aplicação MAUI utiliza o padrão RESTful para comunicação com a API, permitindo realizar operações de criação, consulta e eliminação de categorias. A estrutura modular e escalável do projeto facilita a expansão futura para incluir mais funcionalidades e entidades.

Funcionalidades Principais

1. Gestão de Categorias

A aplicação oferece as seguintes funcionalidades para a tabela Category:

- Consulta de Categorias por ID: Permite recuperar informações detalhadas de uma categoria específica.
- Criação de Novas Categorias: Proporciona uma interface simples para inserir novas categorias na base de dados.
- Eliminação de Categorias: Remove categorias existentes de forma segura e eficiente.

Exemplo de implementação para a criação de uma categoria:

```
1. public async Task<Category> CreateCategoryAsync(Category category)
2. {
3.     var json = JsonConvert.SerializeObject(category);
4.     var content = new StringContent(json, Encoding.UTF8, "application/json");
5.
6.     var response = await _httpClient.PostAsync(_baseUrl, content);
7.     response.EnsureSuccessStatusCode();
8.
9.     var responseJson = await response.Content.ReadAsStringAsync();
10.    return JsonConvert.DeserializeObject<Category>(responseJson);
11. }
12.
```

2. Conexão com a API RESTful

A aplicação utiliza a classe CategoryApiClient para encapsular as chamadas RESTful, garantindo uma separação clara entre a lógica de negócios e a interface gráfica. Esta classe estabelece a conexão com a API através do HttpClient, abstraindo os detalhes técnicos da comunicação.

Exemplo de implementação para consulta de uma categoria por ID:

```
1. public async Task<Category> GetCategoryByIdAsync(long id)
2. {
3.     var response = await _httpClient.GetAsync($"_{baseUrl}/{id}");
4.     if (!response.IsSuccessStatusCode)
5.         return null;
6.
7.     var json = await response.Content.ReadAsStringAsync();
8.     return JsonConvert.DeserializeObject<Category>(json);
9. }
10.
```

3. Interface Gráfica Multiplataforma

A aplicação utiliza o poder do .NET MAUI para criar uma interface responsiva e adaptada a diferentes dispositivos. Através do XAML, foi possível definir um layout que oferece uma experiência de utilizador intuitiva e uniforme, independentemente da plataforma.

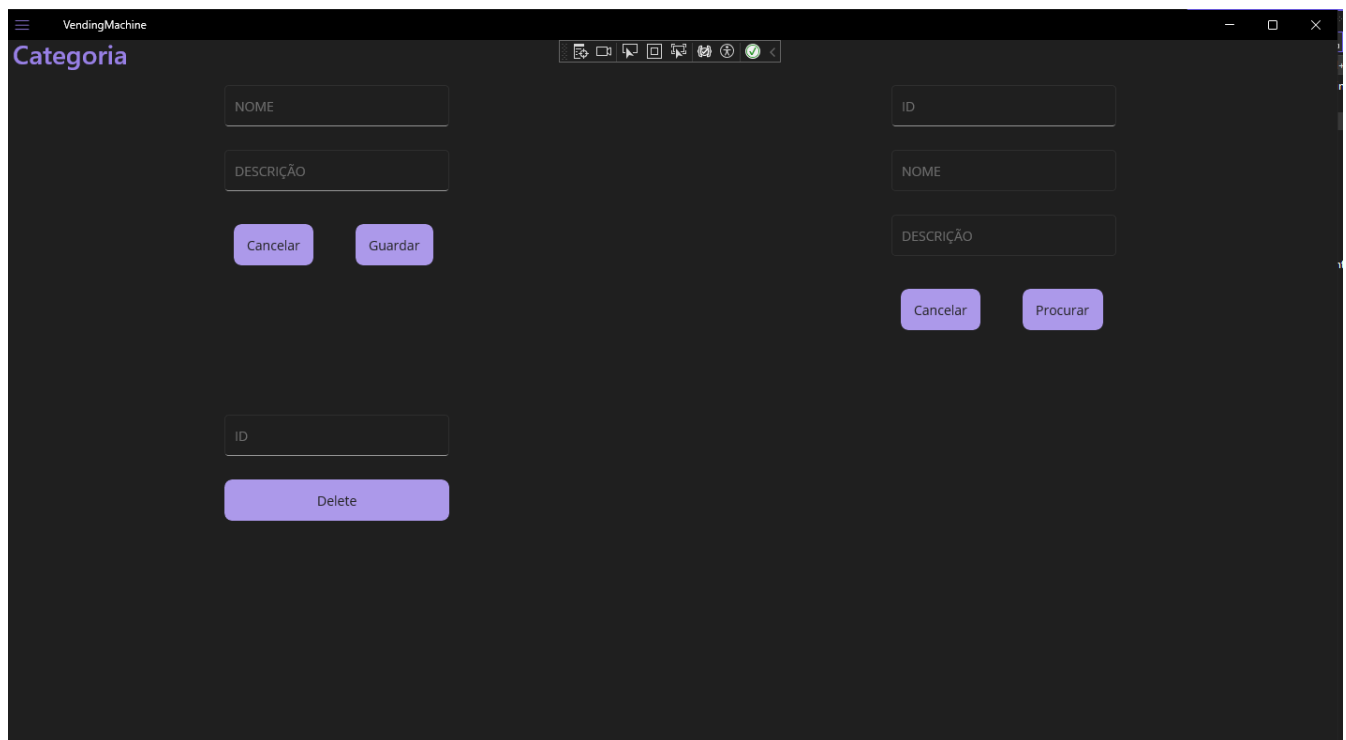


Figura 1 - Ecrã MAUI Categoria

Estrutura da Aplicação

A estrutura da aplicação MAUI segue as melhores práticas para desenvolvimento multiplataforma, dividida em componentes-chave:

- Models: Contém as definições das entidades do sistema. Por exemplo, a classe Category:

```
1. public class Category
2. {
3.     public long Id { get; set; }
```

```

4.     public string Name { get; set; }
5.     public string Description { get; set; }
6. }
7.

```

- CategoryApiClient: Classe responsável pela comunicação com a API RESTful, encapsulando as operações realizadas sobre a tabela Category.
- Interface Principal (MainPage.xaml): A página inicial da aplicação, onde são exibidas as funcionalidades disponíveis para a gestão de categorias.
- MauiProgram.cs: Configuração principal da aplicação, incluindo a definição de fontes e o registo dos serviços utilizados.

Código Desenvolvido

Exemplo de configuração principal da aplicação, onde são definidos os serviços e recursos:

```

1. public static MauiApp CreateMauiApp()
2. {
3.     var builder = MauiApp.CreateBuilder();
4.     builder
5.         .UseMauiApp<App>()
6.         .ConfigureFonts(fonts =>
7.         {
8.             fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
9.             fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
10.        });
11.
12. #if DEBUG
13.     builder.Logging.AddDebug();
14. #endif
15.
16.     return builder.Build();
17. }
18.

```

Exemplo de implementação para eliminação de uma categoria:

```

1. public async Task<bool> DeleteCategoryAsync(long id)
2. {
3.     var response = await _httpClient.DeleteAsync($"({_baseUrl}/{id}");
4.     return response.IsSuccessStatusCode;
5. }

```

Contribuições e Expansão

A aplicação MAUI, embora ainda em fase inicial, demonstra a capacidade de integração eficiente com a API RESTful, servindo como uma base sólida para expandir a funcionalidade do sistema. No futuro, pretende-se:

- Incluir suporte para outras entidades, como Product, User e Inventory.
- Ampliar as operações disponíveis, como a atualização de categorias.

- Melhorar a interface gráfica, incorporando novos elementos visuais e interativos para enriquecer a experiência do utilizador.

A aplicação MAUI representa um passo significativo na criação de um sistema multiplataforma moderno e eficiente, alinhado às melhores práticas de desenvolvimento.

6. Próximos passos

O projeto encontra-se atualmente em fase de desenvolvimento inicial e, até ao momento, não contempla todas as funcionalidades e operações planeadas. Para alcançar os objetivos propostos e garantir um sistema completo, eficiente e seguro, é essencial dar continuidade ao desenvolvimento através das seguintes etapas:

1. Conclusão das Classes e Operações Básicas

O primeiro passo será completar todas as classes e operações relacionadas às entidades do sistema, assegurando que todas as tabelas, como User, Category, Product e Inventory, tenham suas operações CRUD (Create, Read, Update, Delete) devidamente implementadas. Adicionalmente, será necessário desenvolver operações mais complexas que envolvam a interação entre tabelas, como a geração de relatórios de inventário e transações, ou a associação de produtos a categorias.

Estas operações são fundamentais para estabelecer uma base sólida no sistema, permitindo que as APIs ofereçam suporte completo às funcionalidades propostas.

2. Expansão e Conclusão do Frontend

Após a conclusão das operações básicas, o foco será direcionado para o desenvolvimento das aplicações frontend, tanto em consola quanto em MAUI, garantindo que ambas sejam capazes de consumir todas as operações implementadas nas APIs. Os próximos passos incluem:

- Adicionar todas as funcionalidades restantes, como a gestão de utilizadores, produtos e inventário.
- Expandir o menu e a interface gráfica para permitir uma experiência de utilização completa e intuitiva.
- Certificar-se de que ambas as aplicações possam ser utilizadas para o controlo total dos dados do sistema, facilitando a gestão centralizada e eficiente.

Este trabalho no frontend será essencial para fornecer uma experiência completa ao utilizador, com suporte a todas as operações oferecidas pelas APIs.

3. Melhoria da Segurança do Sistema

Por último, será prioritário trabalhar na segurança de toda a aplicação, introduzindo funcionalidades de autenticação e autorização. Entre os elementos a serem implementados estão:

- Sistema de Login: Para garantir que apenas utilizadores autenticados possam aceder à aplicação.
- Gestão de Roles: Adicionar níveis de permissão (e.g., Admin e Operador) para limitar as ações possíveis de acordo com o tipo de utilizador.
- Restrição de Ações: Implementar mecanismos para assegurar que apenas utilizadores autorizados possam realizar operações sensíveis, como criação ou eliminação de registos.

Esta camada adicional de segurança é imprescindível para proteger os dados do sistema e garantir que as operações sejam realizadas de forma controlada e segura.

Conclusão

Com estes próximos passos, pretende-se completar as funcionalidades do sistema e assegurar a sua usabilidade e segurança. A conclusão das operações básicas, a expansão das interfaces frontend e a implementação de mecanismos de segurança permitirão criar uma solução robusta, eficiente e alinhada aos requisitos propostos, garantindo a satisfação dos utilizadores e a gestão eficaz das máquinas de vending.

7. Conclusão

O desenvolvimento deste projeto permitiu criar uma base sólida para a gestão de máquinas de vending, integrando conceitos de sistemas de informação, desenvolvimento de APIs e implementação de serviços web. Apesar de o projeto ainda estar incompleto, os progressos realizados demonstraram a viabilidade da solução e a capacidade de integração de tecnologias modernas, como RESTful APIs, serviços SOAP, e frameworks multiplataforma como .NET MAUI.

Até ao momento, foram desenvolvidas funcionalidades essenciais para a gestão de entidades como Category, Product e User, implementadas em API e consumidas por aplicações frontend. A aplicação em consola permitiu validar as operações básicas e a integração com serviços externos, como a conversão de moedas, enquanto a aplicação em MAUI representa um passo importante na criação de uma interface gráfica moderna e multiplataforma.

Para completar o sistema, será necessário expandir as funcionalidades existentes, assegurar a integração total entre os componentes e introduzir mecanismos avançados de segurança, como autenticação e gestão de permissões. Este projeto evidenciou a importância de uma arquitetura escalável, modular e segura, que poderá ser desenvolvida e refinada para atender às necessidades específicas do cenário.

Por fim, o trabalho realizado até agora contribuiu significativamente para a consolidação de competências técnicas, nomeadamente no desenvolvimento de APIs, consumo de serviços web, integração de tecnologias e design de sistemas de informação. Assim, este projeto não só responde aos requisitos académicos como também serve como um exemplo prático de aplicação de conhecimento em um contexto real.

8. Bibliografia

<https://www.cockroachlabs.com/docs/>

<https://freecurrencyapi.com/docs/currencies>

<https://learn.microsoft.com/en-us/dotnet/framework/wcf/>

<https://learn.microsoft.com/en-us/dotnet/maui/?view=net-maui-9.0>

<https://www.npgsql.org/doc/>

<https://learn.openapis.org/>

<https://docs.github.com/en>

<https://learn.microsoft.com/en-us/dotnet/csharp/>

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>